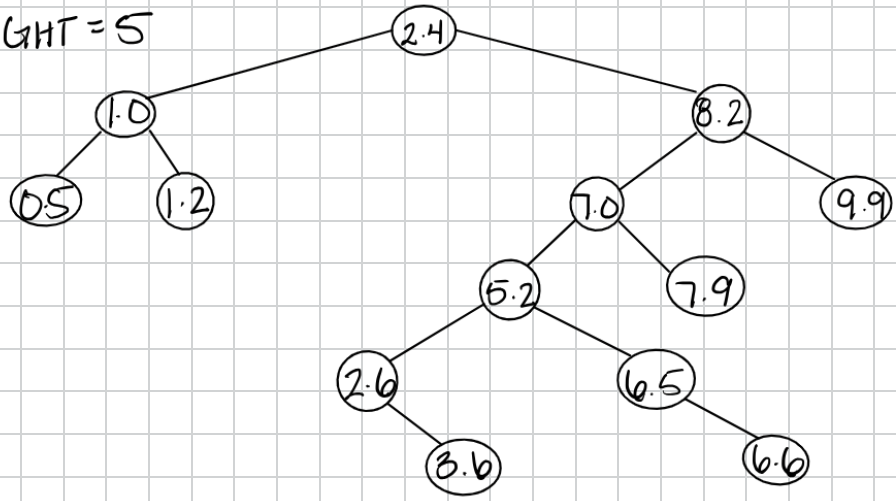
Assignment #6 Trees and Heaps

1.

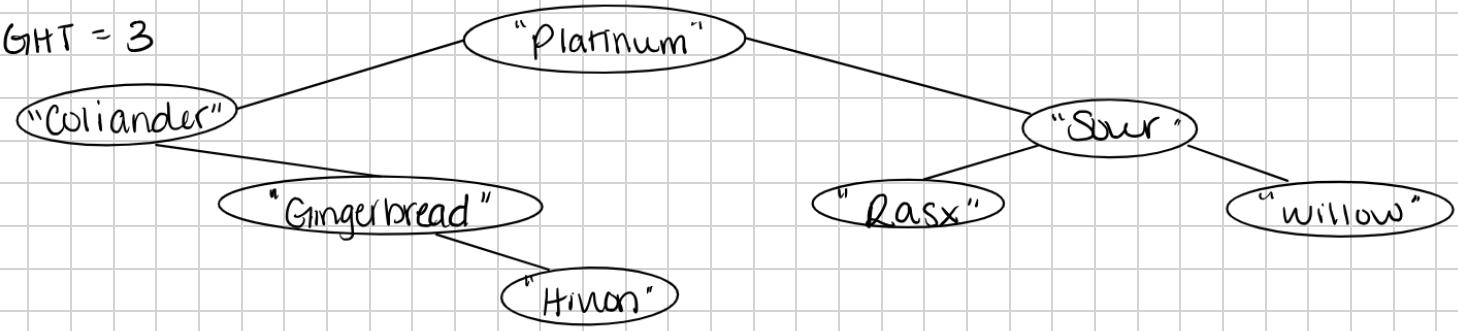
a) [2.4, 8.2, 7.0, 5.2, 1.0, 6.5, 2.6, 3.6, 7.9, 0.5, 6.6, 9.9, 1.2]

HEIGHT = 5



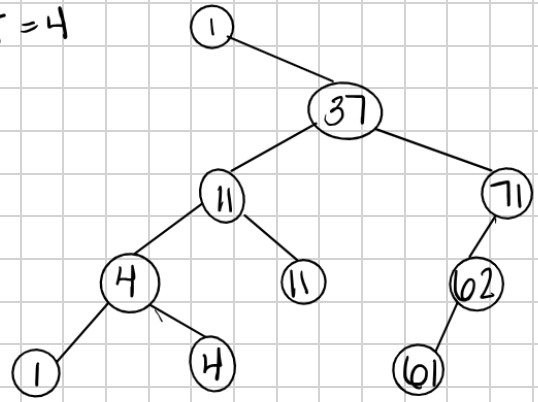
b) ["Platinum", "Coliander", "Sour", "willow", "Rasx", "Gingerbread", "Hinon"]

HEIGHT = 3



c) [1, 37, 11, 71, 62, 4, 61, 1, 11, 4]

HEIGHT = 4

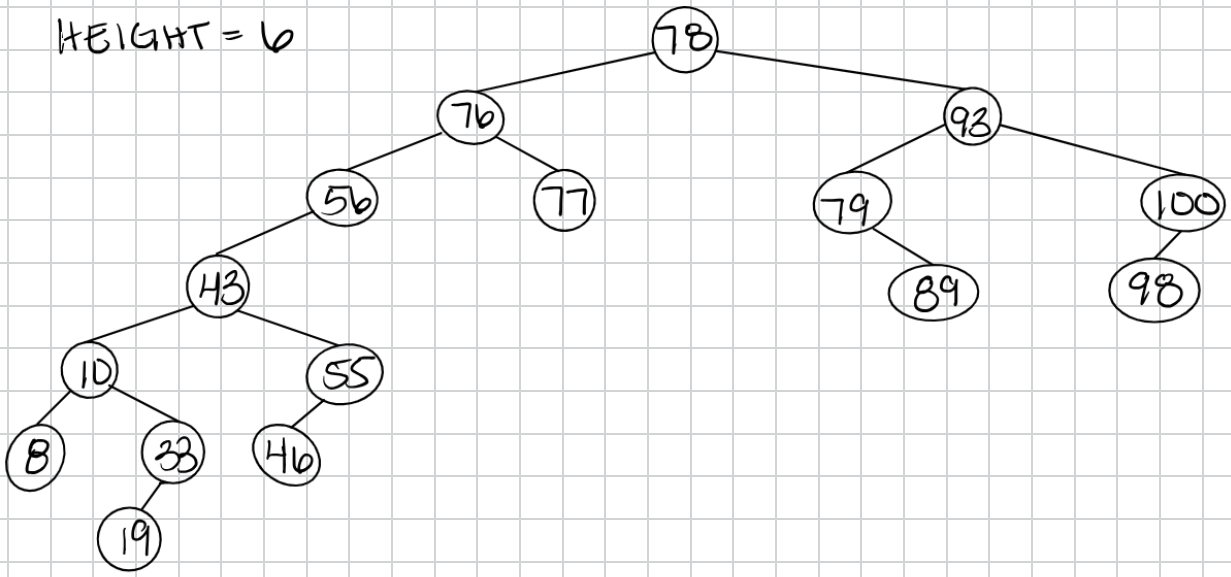


Assignment #6 Trees and Heaps

1.

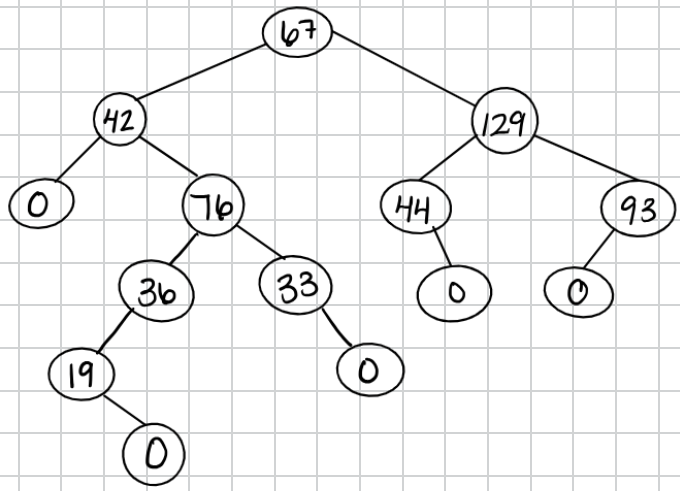
d.) [78, 76, 56, 43, 55, 77, 10, 93, 46, 79, 33, 8, 89, 100, 98, 19]

HEIGHT = 6

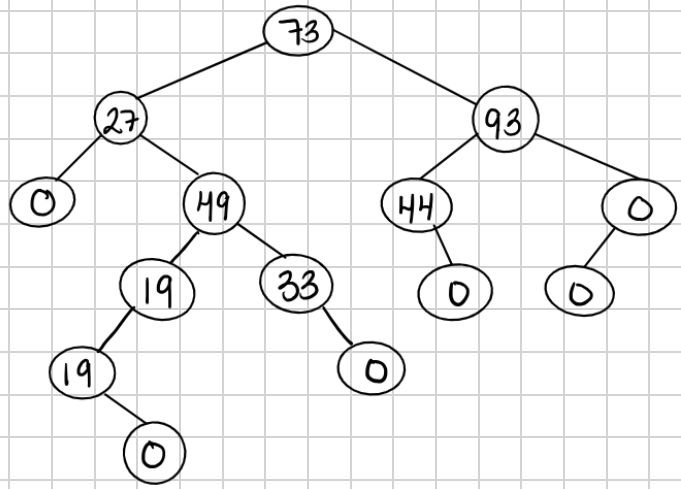


Assignment #6 Trees and Heaps

2. a) Preorder



In order



b.) BST?

Preorder: The preorder tree is not a BST because it violates the ordering property. We see the violation in the left subtree at 76; 76 is greater than 67 and would need to be in the right subtree for this to be be a valid BST. The right subtree also violates the ordering property. 44 should be in the left subtree and 93 in 129's left subtree for this to be valid. An additional violation of the ordering property is all the leaf nodes being 0.

Inorder: The inorder tree is also not a BST because it also violates the ordering property. We see the violation in left subtree and the subtrees of 49. The 19s should be in 27's left subtree and 33 in 49's left subtree. Violations in the right subtree are 44, which should be in 73's left subtree. Again, the leaf nodes being 0 are again a violation, as mostly all are not ordered.

c.) AVL?

Since both trees violate the ordering property and are not BSTs, they cannot be AVLs. This is because an AVL has to satisfy all BST properties to be considered an AVL.

Assignment 6 – Stacks and Queues
Benevento

Question 7: Algorithm Analysis

3. **Time Complexity:**

The frequencyCount method has a time complexity of O(n) in the worst case and Ω(n) in the best case, where n is the number of characters in the string. The frequencyCount method iterates through each character in the input string exactly once. It performs constant operations on each character, checking if it is in the HashMap and updating or adding it.

The buildHuffman method has a time complexity of O(m log m) in worst and best cases, where m is the number of unique characters. We use a priority queue to represent our Huffman tree. This method adds each unique character into the priority queue as a node, and each heap operation takes O(log m) time. The method repeatedly remoces the two smallest nodes and inserts a new combined node, which is O(m log m).

The genCode and buildCodes methods have a best and worst time complexity of O(m) where m is the number of unique characters. The buildCodes method traverses through each node in the Huffman tree once to assign each character a binary code. Since genCode calls the buildCodes method, it also runs for O(m) time.

The encode method has a time complexity of O(n) time and Ω(n), where n is the number of characters in the input string. The method iterates through each character in the input string, performing constant operations. Thus, it runs for O(n) time.

The printStats method has a time complexity of of O(m log m + n), where n is again the number of characters in the input string and m is the number of unique characters. We copy each unique character into a List, O(m), so we can sort the characters, O(m log m). We then repeat this for the code map and also call encode, which adds O(n).

Combining the individual method analysis, we have an overall time complexity of O(n + m log m), where n is the number of characters in the input string and m is the number of unique characters. We have methods that iterate over the entire string, O(n), and methods that build and sort based on the unique characters, O(m log m). Thus, we have a total time complexity of O(n + m log m).

Space Complexity:

The frequencyCount method has a space complexity of O(m) in the worst case and Ω(1) in the best case, where m is the number of unique characters. The method stores each

unique character and its frequency in a HashMap, which grows based on the number of distinct characters in the input string.

The buildHuffman method has a space complexity of O(m) in both the worst and best cases. The priority queue stores one node per unique character, and the resulting Huffman tree contains the number of nodes proportional to m.

The genCode and buildCodes methods have a space complexity of O(m). The codeMap stores one binary code per unique character, and the recursive calls use additional space proportional to the height of the tree, O(m).

The encode method has a space complexity of O(n) and Ω(n), where n is the number of characters in the input string. The method builds a new encoded string whose size grows proportionally with the length of the input.

The printStats method has a space complexity of O(m + n). It creates temporary lists to store and sort the unique characters, which requires O(m) space. It also stores the encoded string, which requires O(n) space.

Combining the individual method analysis, the overall space complexity is O(n + m), where n is the number of characters in the input string and m is the number of unique characters. This is because the algorithm must store both the encoded output and the character mappings.

4. **Time Complexity:**

The findTarget method has a time complexity of O(n) in the worst case and Ω(1) in the best case, where n is the number of elements in the input array. This is because it calls the dfs method starting at the root of the tree.

The dfs method determines the overall runtime. It has a time complexity of O(n) in the worst case. It visits every reachable node once in the array-based tree representation. Each node results in at most two recursive calls (left and right), but each node is processed only once, so the total work is proportional to the number of nodes. Thus, it runs at O(n) time.

In the best case, the target is found at the root and the method returns immediately, giving Ω(1) time.

Overall, the time complexity is O(n).

Assignment 6 – Stacks and Queues
Benevento

Space Complexity:

The space complexity is O(n) in the worst case and Ω(1) in the best case, where n is the number of elements in the input array.

This is because of how deep the algorithm goes into the tree. Each recursive call moves down one level, so the total space used depends on the height of the tree. In the worst case, the tree can become a single long chain, meaning the height is O(n), so the space complexity is O(n). In the best case, the target is found at the root, so the algorithm does not go deeper than one level, Ω(1).

Overall, space complexity is O(n).

5. **Time Complexity:**

The findAllLess method has a time complexity of O(n) in the worst case and Ω(n) in the best case, where n is the number of strings in the input array. This is because the method loops through each element in the array exactly once. For each string, only constant-time operations are performed, such as checking for null, checking if the string is empty, and checking the string length. Therefore, the total time complexity is O(n).

Space Complexity:

The space complexity is O(n) in the worst case and Ω(1) in the best case, where n is the number of strings in the input array. This is because the method creates a result list that stores all strings that satisfy the condition length < x. In the worst case, all strings in the array could meet this condition, so the result list could grow to size n.

In the best case, no strings are added to the result list, so only a constant amount of extra space is used. Thus, space complexity is O(n).

6. **Time Complexity:**

The insert method has a time complexity of O(n) in the worst case and Ω(1) in the best case, where n is the number of nodes in the tree. This is because the method recursively traverses down the tree to find the correct insertion point. In a balanced tree, this would take O(log n), but in the worst case (an unbalanced tree), it is O(n).

The delete method also has a time complexity of O(n) in the worst case and Ω(1) in the best case. This is because it may need to search for the node to delete and, in the case of two children, find the inorder successor.

The inorder method has a time complexity of O(n) in both worst and best cases because it visits every node exactly once.

The inRange method has a time complexity of $O(n^2)$ in the worst case and Ω(1) in the best case. This is because inRangeHelper traverses the entire tree O(n), and for each node it may call delete, which itself can take O(n) time. Therefore, in the worst case, every node requires a deletion and thus, $O(n^2)$.

Overall, the time complexity is $O(n^2)$.

Space Complexity:

The space complexity is O(n) in the worst case and Ω(1) in the best case, where n is the number of nodes in the tree.

This is because the tree stores all inserted nodes, which grows at a rate of O(n) with the number of elements inserted. Also, recursive calls in insert, deleteBST, inorder, and inRangeHelper use space proportional to the height of the tree. In the worst case (an unbalanced tree), this height can become O(n) causing space complexity to be O(n).

Overall, the space complexity is O(n).